

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	Sameer Ahmed G	Reg. No.:	192421154
Section / Batch:	B Tech IT	Date:	01/08/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Write the algorithm and execute the code for the given experiment.
- Follow a well-structured, systematic approach to design and implement an optimal solution.
- Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.

Question Type	Focus Area	Questions
Experiments	Design and Code for Divide and Conquer Algorithms: Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication	Q1

SECTION A – Experiments**Code of Conduct**

I Sameer Ahmed G (192421154) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Sameer Ahmed G

RUBRICS FOR CALCULATION

Experiments					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding ; unable to integrate concepts
Experimental Design	30%	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15%	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10%	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Experimental Design	30%				
Use of Tools	20%				
Analysis of Results	15%				
Documentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1) Evaluate the performance of Strassen's Matrix Multiplication for matrices of varying sparsity levels. Record execution time and compare with standard multiplication. Analyse how sparsity affects computational efficiency. Interpret the results and justify the applicability of Strassen's algorithm.

SIMATS Online Lab
Ask Gemini
School

SIMATS ENGINEERING
SIMATS Online Programming Lab
DAA PROGRAMMING
SAMEER AHMED G 192421154

CO3-AT1-EXPERIMENTS
Back

Language: Python C C++ Java

QUESTIONS
Q1 Matrix Multiplication 10 marks
0/1 answered

Q1 Matrix Multiplication 10 marks

Evaluate the performance of Strassen's Matrix Multiplication for matrices of varying sparsity levels. Record execution time and compare with standard multiplication. Analyse how sparsity affects computational efficiency. Interpret the results and justify the applicability of Strassen's algorithm.

Your Code

```

1 def add_matrix(A, B):
2     n = len(A)
3     C = [[0]*n for _ in range(n)]
4     for i in range(n):
5         for j in range(n):
6             C[i][j] = A[i][j] + B[i][j]
7     return C
8
9 def sub_matrix(A, B):
10    n = len(A)
11    C = [[0]*n for _ in range(n)]
12    for i in range(n):
13        for j in range(n):
14            C[i][j] = A[i][j] - B[i][j]
15    return C

```

Input

stdin input

Output

```

Sparsity: 0 % | Correct: True
Checksums: 4155 4155
Sparsity: 25 % | Correct: True
Checksums: 2364 2364
Sparsity: 50 % | Correct: True
Checksums: 872 872

```

Python Code :

```
def add_matrix(A, B):
    n = len(A)
    C = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            C[i][j] = A[i][j] + B[i][j]
    return C
```

```
def sub_matrix(A, B):
    n = len(A)
    C = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            C[i][j] = A[i][j] - B[i][j]
    return C
```

```
def standard_mul(A, B):
    n = len(A)
    C = [[0]*n for _ in range(n)]
    for i in range(n):
        for k in range(n):
            if A[i][k] != 0:
                aik = A[i][k]
                for j in range(n):
                    C[i][j] += aik * B[k][j]
    return C
```

```
def split_matrix(A):
    n = len(A)
    m = n//2
    A11 = [[0]*m for _ in range(m)]
    A12 = [[0]*m for _ in range(m)]
    A21 = [[0]*m for _ in range(m)]
    A22 = [[0]*m for _ in range(m)]
    for i in range(m):
        for j in range(m):
            A11[i][j] = A[i][j]
            A12[i][j] = A[i][j+m]
            A21[i][j] = A[i+m][j]
            A22[i][j] = A[i+m][j+m]
```

```
return A11, A12, A21, A22
```

```
def join_matrix(C11, C12, C21, C22):  
    m = len(C11)  
    n = m*2  
    C = [[0]*n for _ in range(n)]  
    for i in range(m):  
        for j in range(m):  
            C[i][j] = C11[i][j]  
            C[i][j+m] = C12[i][j]  
            C[i+m][j] = C21[i][j]  
            C[i+m][j+m] = C22[i][j]  
    return C
```

```
def strassen_mul(A, B, cutoff=2):  
    n = len(A)  
  
    if n <= cutoff:  
        return standard_mul(A, B)
```

```
A11, A12, A21, A22 = split_matrix(A)  
B11, B12, B21, B22 = split_matrix(B)
```

```
M1 = strassen_mul(add_matrix(A11, A22), add_matrix(B11, B22), cutoff)  
M2 = strassen_mul(add_matrix(A21, A22), B11, cutoff)  
M3 = strassen_mul(A11, sub_matrix(B12, B22), cutoff)  
M4 = strassen_mul(A22, sub_matrix(B21, B11), cutoff)  
M5 = strassen_mul(add_matrix(A11, A12), B22, cutoff)  
M6 = strassen_mul(sub_matrix(A21, A11), add_matrix(B11, B12), cutoff)  
M7 = strassen_mul(sub_matrix(A12, A22), add_matrix(B21, B22), cutoff)
```

```
C11 = add_matrix(sub_matrix(add_matrix(M1, M4), M5), M7)  
C12 = add_matrix(M3, M5)  
C21 = add_matrix(M2, M4)  
C22 = add_matrix(sub_matrix(add_matrix(M1, M3), M2), M6)
```

```
return join_matrix(C11, C12, C21, C22)
```

```
seed = 1234567  
def rnd():  
    global seed  
    seed = (seed * 1103515245 + 12345) % 2147483648  
    return seed
```

```

def gen_sparse_matrix(n, sparsity_percent, max_val=5):
    A = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            x = rnd() % 100
            if x >= sparsity_percent:
                A[i][j] = (rnd() % max_val) + 1
            else:
                A[i][j] = 0
    return A

```

```

def run_and_count(fn, A, B):
    R = fn(A, B)
    return R

```

```

def matrices_equal(X, Y):
    n = len(X)
    for i in range(n):
        for j in range(n):
            if X[i][j] != Y[i][j]:
                return False
    return True

```

```

def main():
    n = 8
    sparsities = [0, 25, 50, 75, 90]
    cutoff = 2

```

```

    for sp in sparsities:
        A = gen_sparse_matrix(n, sp)
        B = gen_sparse_matrix(n, sp)

```

```

        C1 = standard_mul(A, B)
        C2 = strassen_mul(A, B, cutoff=cutoff)

```

```

        ok = matrices_equal(C1, C2)
        print("Sparsity:", sp, "% | Correct:", ok)

```

```

    checksum1 = 0
    checksum2 = 0
    for i in range(n):
        for j in range(n):

```

```
checksum1 += C1[i][j]
checksum2 += C2[i][j]
print("Checksums:", checksum1, checksum2)

main()
```